МИНИСТЕРСТВО ОБРАЗОВАНИЯ СВЕРДЛОВСКОЙ ОБЛАСТИ

ГАПОУ СО «ЕКАТЕРИНБУРГСКИЙ МОНТАЖНЫЙ КОЛЛЕДЖ»

КУРСОВОЙ ПРОЕКТ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

Первый этап: REST-сервис на FastAPI

По дисциплине «МДК 05.03 Тестирование информационных систем»

Тема: «Создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении, конкурсах профессионального мастерства»

Выполнил: Пеннер Андрей Владимирович

Группа: ИП-41 / ИП-41к (уточнить)

Номер студенческого: _____

Руководитель: _____

Подписи: _____

Оценка: _____

Екатеринбург
2026

ВЕДОМОСТЬ ПРОЕКТА

Таблица 1 — Состав комплекта первого этапа

№	Наименование	Представление
1	Пояснительная записка	DOCX и PDF
2	REST-сервис FastAPI v1/v2	app/
3	Консольный HTTP-клиент	client/
4	Автоматические тесты	tests/
5	Зависимости и развёртывание	requirements*, compose.yaml, scripts/
6	Руководства и спецификации	docs/*.md, OpenAPI, DDL
7	Диаграммы, отчёты и доказательства	docs/diagrams, examples, evidence
8	Исходные материалы задания	docs/source/

Действующие пароли, .env, виртуальное окружение и содержимое пользовательского хранилища в передаваемый архив исходников не включаются. Демонстрационные данные создаются командой --demo. Полный состав проверяется по манифесту архива.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ · · · 4
1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ · · · 5
1.1 Уточнения и критерии приёмки · · · 6
2 АНАЛИТИЧЕСКАЯ ЧАСТЬ · · · 7
2.1 Цели, функции и задачи системы · · · 7
2.2 Описание предметной области · · · 8
2.3 Пользователи и разграничение доступа · · · 9
2.4 Внутренняя архитектура · · · 11
2.5 Проектные решения по тестированию · · · 12
3 ПРОЕКТНАЯ ЧАСТЬ · · · 13
3.1 Стек технологий · · · 13
3.2 Модель и ограничения базы данных · · · 14
3.3 Файловая структура и назначение модулей · · · 18
3.4 Алгоритм записи на олимпиаду · · · 19
3.4.1 Подготовка, отзыв и публикация протокола · · · 21
3.4.2 Аутентификация и отзыв токенов · · · 22
3.5 Документация REST API · · · 24
3.5.1 Примеры запросов и обработка ответа · · · 26
3.5.2 Клиент первого этапа и граница с Django · · · 27
3.5.3 Руководство по развёртыванию · · · 28
3.5.4 Хранение данных и резервное копирование · · · 29
3.6 Результаты тестирования · · · 30
4 ГЛОССАРИЙ · · · 32
5 СПИСОК ИСТОЧНИКОВ · · · 33
ЗАКЛЮЧЕНИЕ · · · 34
Приложение 1 — Реестр дефектов и ограничений · · · 35
Приложение 2 — Тестовые сценарии · · · 36
Приложение 3 — Матрица трассировки · · · 40
Приложение 4 — Основные листинги · · · 41
Приложение 5 — Запросы и примеры результатов · · · 46
Приложение 6 — Рекомендуемая литература · · · 50

ВВЕДЕНИЕ

Участие студентов в олимпиадах и конкурсах профессионального мастерства относится к результатам учебной и внеучебной деятельности колледжа. Для каждого мероприятия необходимо хранить положение, даты проведения, список участников, полученные баллы и итоговые результаты. Если эти сведения ведутся в отдельных таблицах и документах, их приходится повторно переносить в протоколы и годовые отчёты. При этом легко допустить расхождение между списком участников и опубликованными итогами.

Актуальность проекта связана с необходимостью упорядочить эти сведения и разграничить ответственность пользователей. Студенту нужен доступ к своим заявкам и результатам, преподавателю — к подготовке документов своего мероприятия, администратору — к учётным записям и публикации проверенных протоколов. Гостю достаточно открытых сведений без доступа к редактированию и личным данным учётных записей.

Объект разработки — обработка сведений об участии студентов в олимпиадном движении и конкурсах профессионального мастерства. Предмет разработки — REST-сервис, который хранит эти сведения в PostgreSQL и предоставляет операции через HTTP. Основанием служит персональное техническое задание, выданное Пеннер Андрею Владимировичу [1].

Цель первого этапа — разработать и проверить самостоятельный REST-сервис на FastAPI. Для достижения цели требуется определить сущности и права пользователей; реализовать обработку заявок, документов и результатов; подготовить отчёты Word и Excel; проверить положительные и отрицательные сценарии; составить документацию по установке и использованию.

Работа ограничена первым этапом со сроком до 15 декабря 2026 года. Разработка Django-приложения относится ко второму этапу, срок которого установлен до 1 мая 2027 года. В записке рассматриваются техническое задание, аналитические решения, устройство сервиса, результаты испытаний и порядок эксплуатации. Исходные материалы и правила оформления приведены в [1–5].

1 ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Полная тема проекта: «Создание и тестирование АРМ для обработки информации об участии студентов в олимпиадном движении, конкурсах профессионального мастерства». Разрабатываемый сервис предназначен для ввода, хранения и обработки сведений об олимпиадах, участниках и результатах. Он должен обеспечивать систематизацию информации и формирование отчётности.

Гость просматривает список олимпиад, отбирает их по году, получает карточку, положение, задания прошлых лет и опубликованные протоколы. Студент входит в систему, изменяет собственный профиль, подаёт заявки на несколько мероприятий и просматривает свои опубликованные результаты.

Администратор создаёт преподавателей, редактирует существующие учётные записи, заполняет карточки мероприятий, назначает ответственных и публикует подготовленные протоколы. Ему доступны протокол выбранной олимпиады и годовой отчёт. Преподаватель создаёт студентов, ведёт документы и результаты своей олимпиады, готовит протокол и при необходимости отзывает его до публикации.

В карточке студента хранятся ФИО, дата рождения, электронная почта, логин, учебное заведение, уровень образования, курс и специальность. Пароль принимается при создании, но в открытом виде не сохраняется. Для мероприятия задаются название, описание, ответственный преподаватель и даты. В заявке связываются студент и олимпиада, а после проверки работ сохраняются баллы и итоговая категория.

Обязательные технические условия первого этапа: FastAPI, SQLAlchemy, клиент-серверная PostgreSQL, JWT, версии v1/v2 и отдельная автоматическая документация. Для основных операций предусмотрены HTTP-методы GET, POST, PUT/PATCH и DELETE. Критерии приёмки включают выполнение сценария от заявки до публикации, создание настоящих DOCX/XLSX и проверку ограничений доступа [2].

1.1 Уточнения и критерии приёмки

В исходном перечне категорий отдельно не указан студент, хотя далее описаны действия зарегистрированного участника и создание студенческих учётных записей. Поэтому для участника принята роль student. Самостоятельная публичная регистрация не добавлена: студента создаёт преподаватель или администратор, преподавателя — администратор.

Задание не содержит общих порогов для назначения победителя и призёра. Категория результата вводится ответственным преподавателем по положению конкретной олимпиады. Система проверяет допустимое значение категории и корректность баллов, но не заменяет решение организатора произвольной формулой.

Приняты три состояния протокола: draft — формируется, prepared — подготовлен, published — опубликован. В prepared результаты и заявки заблокированы. Исправление возможно после отзыва в draft. Опубликованный протокол не отзывается, поскольку в ТЗ отзыв разрешён только для подготовленных протоколов.

Критериями приёмки считаются: сохранение информации после повторного запуска; отсутствие доступа преподавателя к чужому мероприятию; отсутствие публичных результатов до публикации; корректное отображение собственных результатов; создание читаемых Word/Excel; успешные автоматические проверки. Матрица требований и реализаций содержится в REQUIREMENTS.md, а связь с тестами — в TESTING.md.

2 АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1 Цели, функции и задачи системы

Главная задача системы — хранить согласованный набор сведений о мероприятиях и участии студентов. Данные вводятся один раз и используются при просмотре, публикации и выгрузке отчётов. Повторное ручное составление нескольких несвязанных таблиц заменяется запросами к общей базе.

Функции распределены между модулями авторизации, учётных записей, олимпиад, участия, протоколов и документов. Такое разделение соответствует действиям пользователей: учётная запись определяет исполнителя, карточка описывает мероприятие, заявка связывает его со студентом, протокол содержит итог, а вложения хранят положение и бланки.

В первом этапе роль пользовательского интерфейса выполняют Swagger и консольный requests-клиент. Оба обращаются к тем же HTTP-маршрутам и получают одинаковые ограничения доступа. При разработке второго этапа Django сможет использовать этот контракт, не обращаясь к таблицам REST-сервиса напрямую.

К задачам обработки относятся фильтрация по году, поиск и пагинация v2, контроль уникальности заявки, проверка полноты протокола, разграничение доступа и подготовка файлов. Удобство сопровождения обеспечивается небольшими функциями, явными проверками и отдельными файлами классов. Сложные уровни абстракции, очереди заданий и распределённая архитектура для этого объёма не требуются.

2.2 Описание предметной области

Мероприятие имеет одного ответственного преподавателя и может иметь много участников. Студент может участвовать в нескольких мероприятиях. Такая связь «многие ко многим» раскрыта через таблицу participations. В ней находятся баллы и результат конкретного студента на конкретной олимпиаде.

Протокол не выделен в отдельную таблицу: у мероприятия существует один текущий набор результатов и один статус. Это уменьшает число сущностей без потери функций первого этапа. История версий протокола и повторное опубликование исправлений не входят в выданное ТЗ.

Документ относится к одной олимпиаде и имеет вид: положение, задание, сертификат или грамота. Любое число записей certificate и award позволяет прикрепить несколько разных бланков. Физические файлы находятся в storage, а в базе — их метаданные. Замена файла сохраняет связь с мероприятием и меняет содержимое через новый случайный путь.

При анализе использованы персональное ТЗ, методические материалы и учебные примеры запросов [1–5]. Опросы сотрудников колледжа и реальные результаты олимпиад не предоставлялись и не заявляются проведёнными. Демонстрационные данные вымышлены, соответствуют требуемой структуре и содержат более 50 записей на каждую основную сущность.

2.3 Пользователи и разграничение доступа

Гость не имеет учётной записи и получает только общедоступные сведения. Студент работает со своим профилем и своими заявками. Преподаватель создаёт студенческие аккаунты и отвечает за мероприятия, где его идентификатор указан в teacher_id. Администратор обслуживает всю систему и выполняет публикацию.

Аутентификация подтверждает личность по логину и паролю, затем по JWT. Авторизация отдельно проверяет роль и принадлежность объекта. Недостаточно проверить только роль teacher: иначе любой преподаватель получил бы возможность менять результаты чужой олимпиады. Поэтому общая функция check_owner сравнивает teacher_id и id пользователя.

Для профиля используется отдельная входная схема без полей role, isActive и tokenVersion. Неизвестные поля отклоняются. Для публичного протокола выбираются ФИО, баллы и результат; email, дата рождения и хеш пароля в ответ не включаются. Ограничения проверяются на сервере и действуют одинаково для Swagger, requests и будущего Django-клиента.

На рисунке 1 показаны варианты использования для четырёх актёров. Стрелки зависимостей между прецедентами не введены, поскольку основные операции представлены самостоятельными вариантами использования. Доступ к общим публичным операциям сохраняется и у зарегистрированных пользователей.

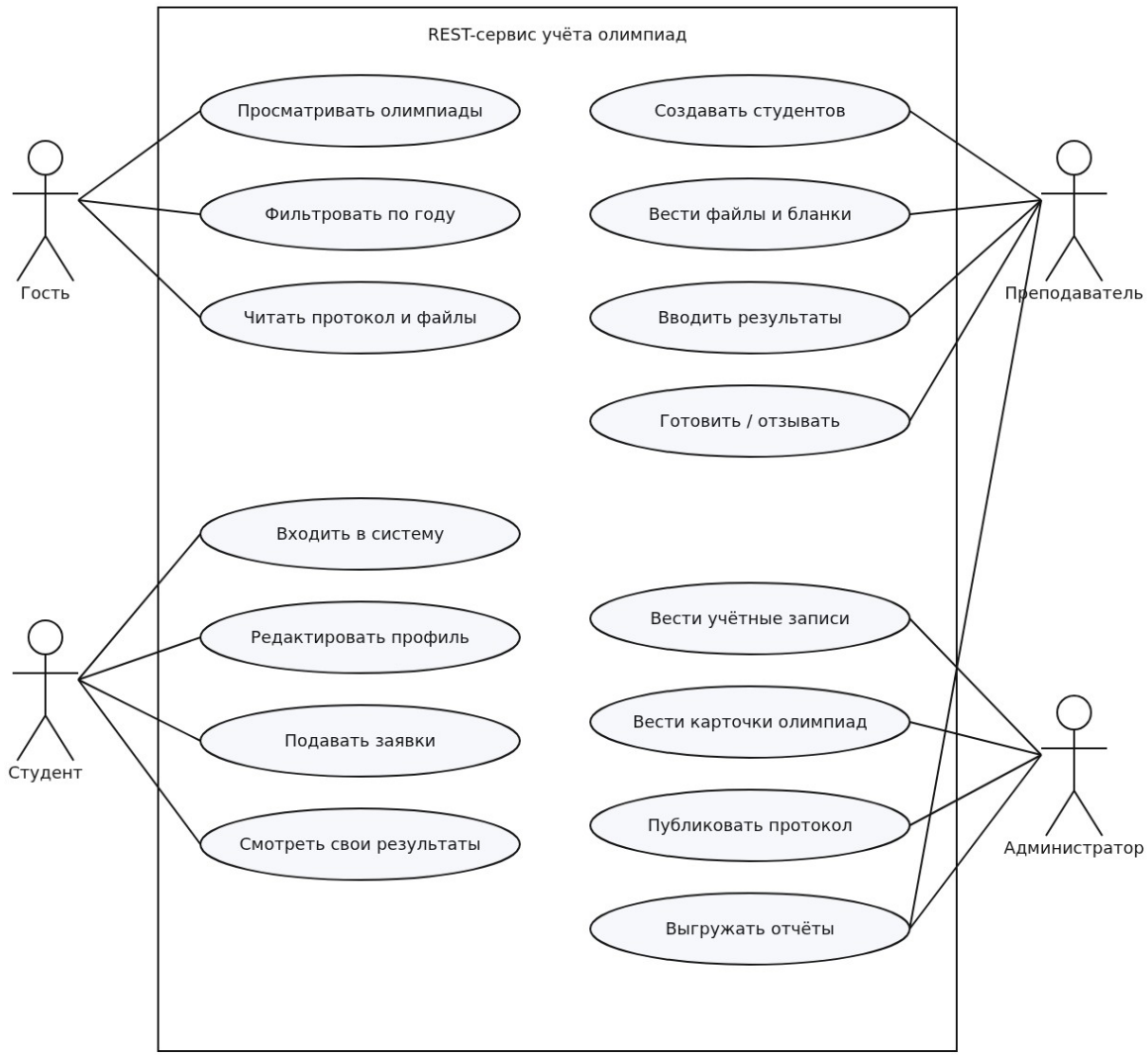


Рисунок 1 — Варианты использования REST-сервиса

2.4 Внутренняя архитектура

Структура приложения показана на рисунке 2. FastAPI получает HTTP-запрос и выполняет проверку тела через Pydantic. Затем обработчик получает сессию SQLAlchemy и текущего пользователя. Повторяемые проверки вынесены в services.py, операции безопасности — в security.py, а построение файлов — в reports.py.

Модели SQLAlchemy описывают четыре таблицы. Они не передаются клиенту напрямую без выбора полей: для основных объектов применяются выходные схемы, для протоколов и заявок явно составляются словари. Это позволяет отделить физическую структуру таблицы users от открытого ответа API.

Сессия БД создаётся на запрос и закрывается после его завершения. При нарушении уникальности выполняется rollback и возвращается 409. Изменения статуса, заявки и результата используют блокировку строки олимпиады. Все версии работают с одной базой; номер API не обозначает отдельную копию данных.

Модульный подход остаётся простым: маршруты содержат обычные функции, без универсальных репозиториев и метапрограммирования. Классы нужны там, где их требует библиотека: для ORM-моделей и схем данных. Каждая такая сущность находится в отдельном файле [6–8].

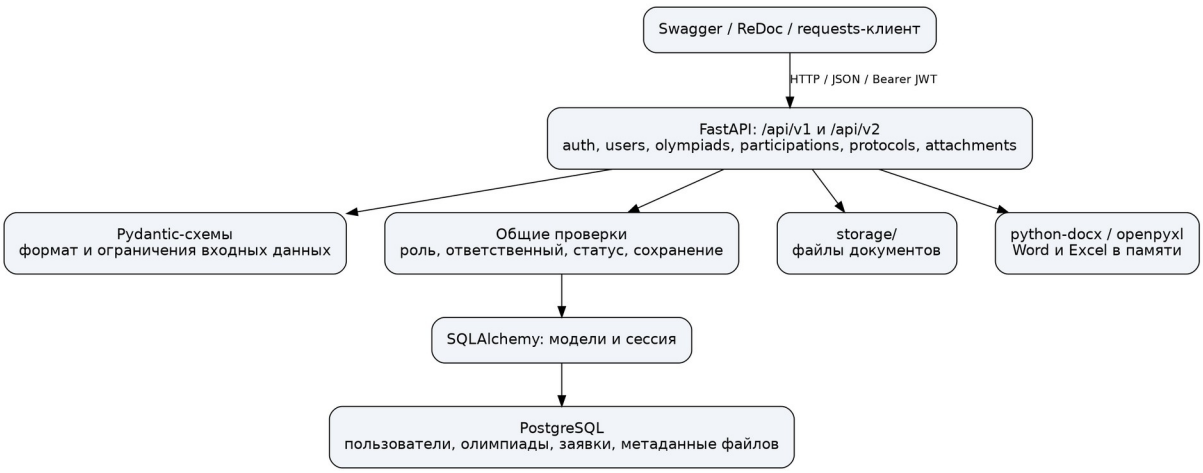


Рисунок 2 — Структура приложения и движение данных

2.5 Проектные решения по тестированию

Проверки должны обнаруживать ошибки в действиях и правах, а не только подтверждать запуск процесса. Для этого выбран pytest, FastAPI TestClient и отдельная PostgreSQL-база. Тесты создают минимальный набор аккаунтов и событий, выполняют HTTP-запросы и проверяют коды, структуру ответа и сохранённое состояние [14].

Положительные случаи дополнены отрицательными: неверные пароли и токены, отсутствующие записи, повторные заявки, отрицательные баллы, неверные даты, чужой преподаватель и попытка публикации неподготовленного протокола. Для файлов проверяются расширение, сигнатура, лимит, UTF-8, замена и удаление. Word и Excel повторно открываются соответствующими библиотеками.

Каждый тест использует внешнюю транзакцию и savepoint. Вызов commit внутри маршрута не делает тестовые данные постоянными: после теста внешняя транзакция откатывается. Каталог загрузок заменяется временным. Тестовая БД имеет имя с суффиксом _test, что дополнительно препятствует случайному запуску на рабочей базе.

Критерий завершения — отсутствие падений выбранного набора и покрытие маршрутов не ниже 80%. Отдельно выполняется HTTP-сценарий через настоящий Uvicorn. Матрица требований, набор сценариев и ограничения испытаний приведены в приложениях 1–3. Нагрузочные и Django UI-проверки на этом этапе не проводились.

3 ПРОЕКТНАЯ ЧАСТЬ

3.1 Стек технологий

Python 3.12 выбран как проверенная версия среды. FastAPI 0.115.12 предоставляет маршруты, зависимые параметры, проверку через Pydantic и OpenAPI. SQLAlchemy 2.0.41 связывает классы с таблицами PostgreSQL. Драйвер psycopg 3.2.9 обеспечивает подключение к клиент-серверной СУБД [6–9].

PyJWT 2.10.1 применяется для подписанных токенов HS256. PBKDF2 реализован средствами стандартной библиотеки hashlib: дополнительный пакет для хранения паролей не нужен. python-multipart разбирает формы авторизации и загрузки. python-dotenv читает локальные настройки, не помещая их в исходный код.

Для Word используется python-docx 1.2.0, для Excel — openpyxl 3.1.5 [12, 13]. Библиотека requests 2.32.4 нужна учебному клиенту и HTTP-проверке [5, 10]. Тестовые зависимости — pytest, pytest-cov и httpx. Точные версии, включая транзитивные, сохранены в requirements-lock.txt.

Используется синхронная SQLAlchemy-сессия и обычные функции def. Для небольшого учебного сервиса это проще для чтения и отладки. База не заменена встроенной SQLite. Docker Compose запускает PostgreSQL отдельно, при этом само Python-приложение может работать в venv на компьютере студента.

3.2 Модель и ограничения базы данных

Физическая модель содержит users, olympiads, participations и attachments. На рисунке 3 показаны первичные и внешние ключи. Словарь полей приведён в следующих таблицах; типы сверены с фактической PostgreSQL-базой. Полный DDL находится в schema.sql.

Учётная запись хранит общие сведения и роль. Для студента обязательны учебные поля; для преподавателя они могут быть пустыми. Обязательность зависит от роли и проверяется схемой создания. Логин и email уникальны. Поле created_by описывает создателя записи, token_version используется для отзыва токенов.

Олимпиада ссылается на ответственного преподавателя. В заявке есть уникальная пара olympiad_id/student_id. Именно ограничение БД препятствует дублированию при одновременных запросах. Внешние ключи не допускают ссылок на отсутствующие записи. Удаление олимпиады с данными блокируется приложением и связями.

При формировании отчёта баллы хранятся как Numeric(8,2), а не как двоичное число с плавающей точкой. В JSON ответ преобразуется к числу для удобства клиента. Категории результата, роли и переходы статусов проверяются приложением; не все правила продублированы SQL CHECK. Это различие отражено в словаре данных.

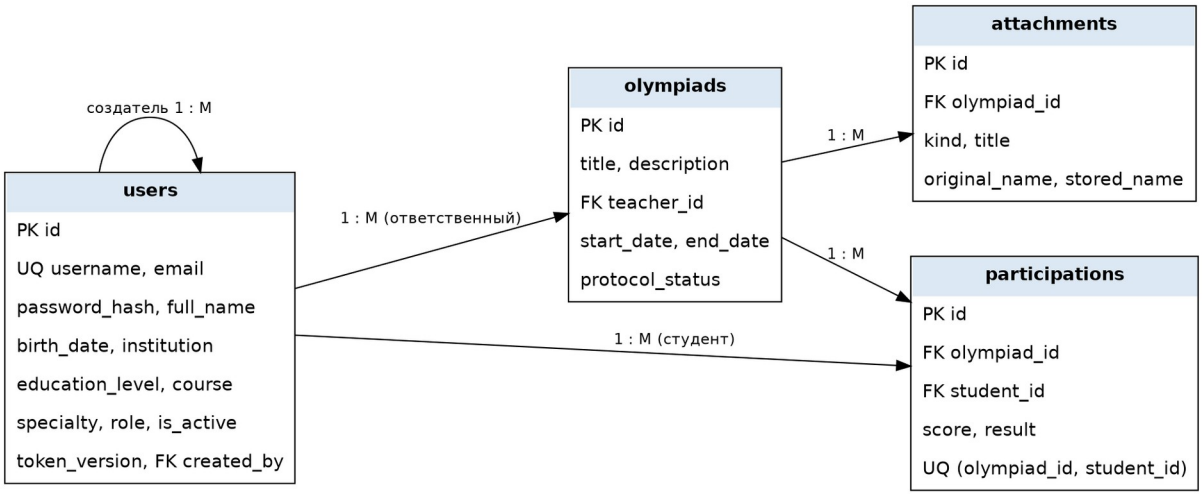


Рисунок 3 — Физическая модель данных PostgreSQL

Состав и типы столбцов получены из фактической PostgreSQL БД через SQLAlchemy Inspector.

PK — первичный ключ; FK — внешний ключ; UQ — уникальность. Проверки ролей и большинства входных полей выполняются схемами Pydantic и обработчиками API; ограничения БД перечислены отдельно.

users

Таблица 2 — Словарь данных: users

Ключ	**Поле**	**Тип PostgreSQL**	**NULL**	**Назначение**
PK	id	INTEGER	нет	Идентификатор аккаунта
UQ	username	VARCHAR(50)	нет	Уникальный логин, 3–50 символов
—	password_hash	VARCHAR(256)	нет	PBKDF2-хеш; исходный пароль не хранится
—	full_name	VARCHAR(150)	нет	Фамилия, имя, отчество
—	birth_date	DATE	да	Дата рождения; для студента обязательна
UQ	email	VARCHAR(254)	нет	Уникальная электронная почта
—	institution	VARCHAR(200)	нет	Учебное заведение
—	education_level	VARCHAR(100)	нет	Уровень образования
—	course	INTEGER	да	Курс обучения, 1–6
—	specialty	VARCHAR(150)	нет	Код и название специальности
—	role	VARCHAR(20)	нет	admin / teacher / student
—	is_active	BOOLEAN	нет	Признак действующего аккаунта
—	token_version	INTEGER	нет	Версия токенов для отзыва сеансов
FK → users.id	created_by	INTEGER	да	Кто создал аккаунт; допускается NULL для начального администратора

olympiads

Таблица 3 — Словарь данных: olympiads

Ключ	Поле	Тип PostgreSQL	NULL	Назначение
PK	id	INTEGER	нет	Идентификатор мероприятия
—	title	VARCHAR(200)	нет	Название
—	description	TEXT	нет	Описание
FK → users.id	teacher_id	INTEGER	нет	Ответственный действующий преподаватель
—	start_date	DATE	нет	Дата начала и источник года для фильтра
—	end_date	DATE	нет	Дата окончания, не раньше начала
—	protocol_status	VARCHAR(20)	нет	draft / prepared / published

participations

Таблица 4 — Словарь данных: participations

Ключ	Поле	Тип PostgreSQL	NULL	Назначение
PK	id	INTEGER	нет	Идентификатор заявки
FK → olympiads.id	olympiad_id	INTEGER	нет	Ссылка на мероприятие
FK → users.id	student_id	INTEGER	нет	Ссылка на студента
—	score	NUMERIC(8, 2)	да	Баллы 0…999999.99, два десятичных знака; NULL до ввода
—	result	VARCHAR(20)	да	winner / prize / participant; NULL до ввода

attachments

Таблица 5 — Словарь данных: attachments

Ключ	Поле	Тип PostgreSQL	NULL	Назначение
PK	id	INTEGER	нет	Идентификатор документа
FK → olympiads.id	olympiad_id	INTEGER	нет	Владелец документа — мероприятие
—	kind	VARCHAR(20)	нет	regulation / task / certificate / award
—	title	VARCHAR(150)	нет	Отображаемое название
—	original_name	VARCHAR(200)	нет	Исходное безопасное имя для скачивания
—	stored_name	VARCHAR(100)	нет	Случайное имя физического файла, не выдаётся API

Связи и ограничения

users → olympiads: один преподаватель отвечает за много мероприятий. users → participations: один студент имеет много заявок. olympiads → participations и olympiads → attachments: связи один-ко-многим. users.created_by ссылается на users.id.

В БД действуют уникальности username, email и пары (olympiad_id, student_id), CHECK end_date ≥ start_date, CHECK score ≥ 0, NOT NULL и внешние ключи. Значения ролей, категории результата, верхняя граница баллов, активность ответственного, обязательность учебных полей и переходы статусов проверяются приложением. Ошибки целостности откатываются и возвращают 409.

Удаление олимпиад с зависимостями блокируется. Файловое содержимое хранится вне БД: attachments содержит только метаданные. Схема соответствует третьей нормальной форме в принятом объёме: данные студента не дублируются в каждой заявке; название мероприятия не дублируется в результатах; справочные роли и типы имеют небольшой фиксированный набор и не вынесены в отдельные таблицы.

3.3 Файловая структура и назначение модулей

Корень содержит README, файлы зависимостей, Compose и пример настроек. Каталог app объединяет сервер, client — отдельный учебный HTTP-клиент, tests — автоматические проверки, scripts — инициализацию и вспомогательные действия, docs — документацию и доказательства проверок. В storage находятся только загруженные документы, а не исходный код.

Маршруты api/auth.py отвечают за вход и профиль, users.py — за учётные записи, olympiads.py — за карточки и версии списка, participations.py — за заявки и баллы, protocols.py — за состояния и выгрузки, attachments.py — за файлы. В models и schemas каждая описанная сущность размещена в своём файле.

main.py создаёт два приложения API и подключает общие маршруты. Для v2 отдельно включается маршрут списка с поиском и пагинацией. Подключение через include_router сохраняет работу зависимостей и позволяет переопределить подключение к БД в тестах. Полные снимки схем экспортированы в openapi-v1.json и openapi-v2.json.

В client/api находятся небольшие функции запросов в стиле презентации. Они возвращают Response, чтобы вызывающий код мог проверить status_code и затем вызвать json(). Для бинарного экспорта используется content, а для ответа DELETE 204 не выполняется попытка прочитать JSON.

3.4 Алгоритм записи на олимпиаду

Схема записи приведена на рисунке 4. Сервер получает Bearer-токен и проверяет существование активного студента. Роль teacher или admin не даёт права записать себя как участника. Мероприятие ищется по идентификатору; отсутствие записи даёт 404.

До добавления заявки блокируется строка олимпиады. Проверяется статус draft и дата окончания. Условие даты позволяет подать заявку в последний день включительно. Затем создаётся Participation с пустыми баллами и результатом. Успех возвращает 201 и идентификатор заявки.

Если такая пара студент–олимпиада уже существует, ограничение UNIQUE вызывает ошибку целостности. Общая функция save откатывает транзакцию и преобразует ошибку в 409. Проверка только предварительным SELECT была бы недостаточна: два одновременных запроса могли бы оба не найти заявку до вставки.

Отмена заявки выполняется отдельным DELETE. Студент может удалить только свою заявку, преподаватель — в своей олимпиаде, администратор — любую при draft. После подготовки протокола состав участников фиксируется. Эти правила проверяются положительными и отрицательными тестами.

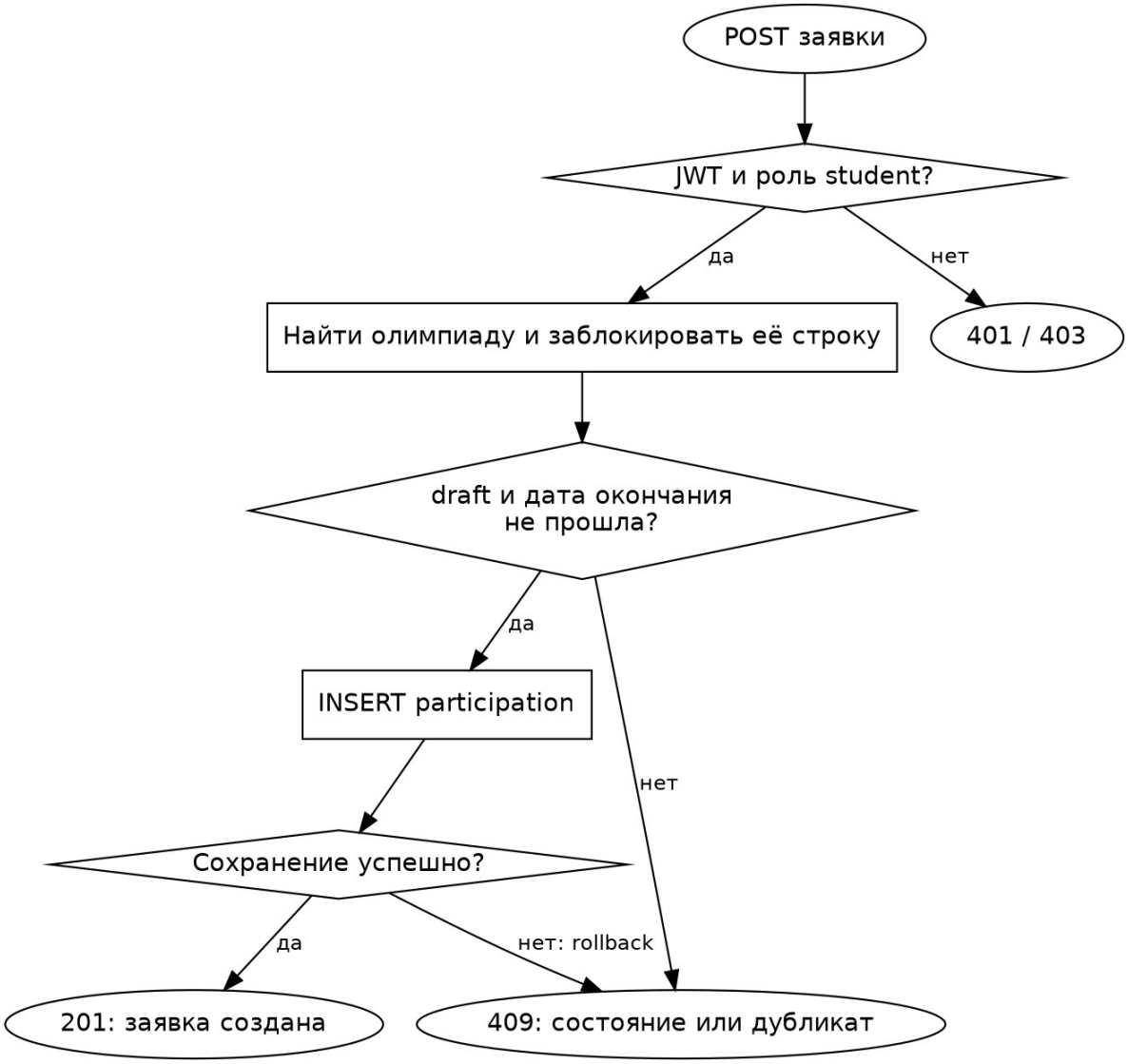


Рисунок 4 — Алгоритм обработки заявки

3.4.1 Подготовка, отзыв и публикация протокола

Модель состояний показана на рисунке 5. Исходное состояние — draft. В нём преподаватель заполняет баллы и категорию результата каждого участника. Список сортируется по убыванию баллов, однако это не назначает призовое место автоматически.

При переходе draft → prepared проверяются ответственность пользователя, наступление даты окончания, наличие участников и заполненность всех результатов. Только после этого сохраняется prepared. Повторный такой же переход и попытка пропустить подготовку приводят к 409.

Ответственный может отозвать prepared в draft. Тогда редактирование становится доступно снова. Переход prepared → published выполняет только администратор. При published публичный маршрут начинает возвращать результаты, а персональный список студента перестаёт скрывать баллы.

Готовность Word/Excel не равнозначна публикации: преподаватель может скачать черновую выгрузку для проверки. Её заголовок содержит текущий статус. Гость получает протокол только через публичный маршрут, который до публикации возвращает 404. Факт появления результата после публикации проверен HTTP-сценарием.

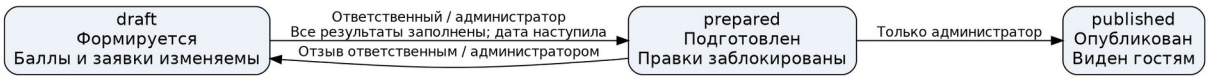


Рисунок 5 — Состояния и переходы протокола

3.4.2 Аутентификация и отзыв токенов

Схема проверки защищённого запроса приведена на рисунке 6. Пароль проверяется через PBKDF2-HMAC-SHA256, соль генерируется отдельно для каждого хеша. Используется 600000 итераций. Результат сравнивается через hmac.compare_digest. Исходный пароль и JWT-ключ в ответы API не включаются.

JWT содержит sub, iat, exp и ver. Сервер принимает только HS256 с настроенным ключом и проверяет обязательные поля [11]. После проверки подписи из PostgreSQL читается пользователь. Неактивный аккаунт, отсутствующая запись или несовпадающий token_version дают 401.

Для выхода token_version увеличивается. По той же причине смена пароля и блокировка отзывают ранее выданные токены. Это добавляет проверку БД к каждому защищённому запросу, зато обеспечивает понятный механизм немедленного отзыва без отдельной таблицы сеансов.

Роль и ответственность проверяются после аутентификации. Поэтому корректный JWT чужого преподавателя не открывает доступ к мероприятию. 401 означает необходимость повторного входа, 403 — отсутствие права при известной личности. Срок токена по умолчанию равен 60 минутам.

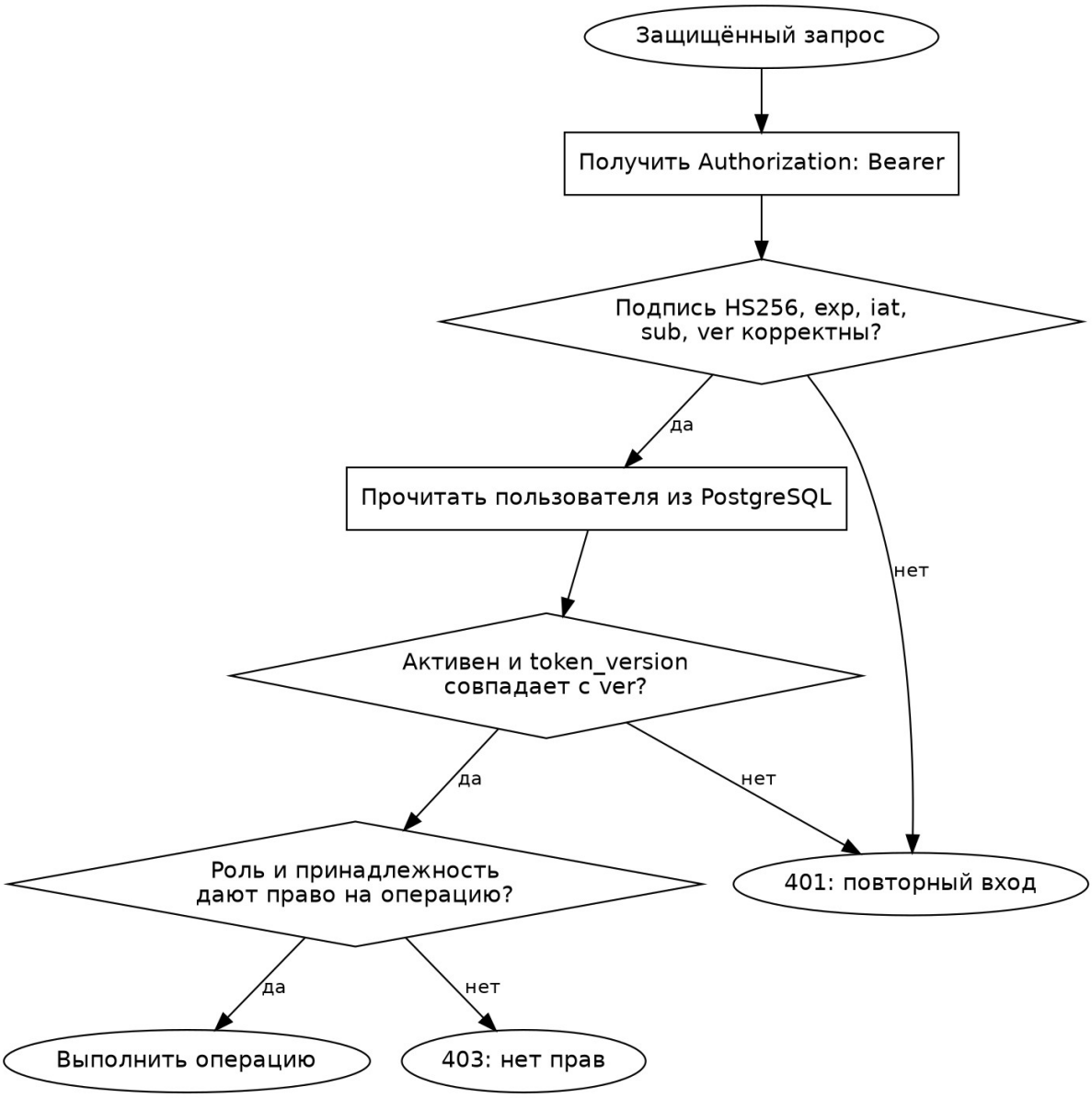


Рисунок 6 — Проверка JWT и прав пользователя

3.5 Документация REST API

REST-сервис использует JSON и стандартные HTTP-методы. Коллекции названы во множественном числе. Токен передаётся заголовком Authorization, а не параметром URL. Поля JSON оформлены camelCase; Python-поля моделей имеют snake_case. Такое преобразование сосредоточено в базовой схеме, а не повторяется в каждом обработчике [5, 15].

v1 предоставляет обычный массив олимпиад. v2 возвращает items, total, page, pageSize, добавляет поиск по названию и ограничение размера страницы до 100. Все остальные маршруты и правила доступа общие. Это позволяет показать развитие контракта без искусственного сохранения ошибок в старой версии.

Swagger и ReDoc доступны отдельно для каждой версии. Скрипты документации сохранены локально в app/static, поэтому при открытии страницы не требуется CDN. Полные схемы и примеры тел входят в API.md и OpenAPI JSON. Ниже приведены группы операций; последовательность действий по ролям вынесена в USER_GUIDE.md.

В таблице 6 приведён состав операций версии v1. Версия v2 заменяет только GET коллекции олимпиад.

Таблица 6 — Операции REST-сервиса

Метод	**Путь**	**Назначение**
POST	/auth/token	Получить JWT-токен
GET	/users/me	Посмотреть свой профиль
PATCH	/users/me	Изменить свой профиль
PUT	/users/me/password	Сменить пароль и отозвать старые токены
DELETE	/auth/session	Завершить все свои сеансы
GET	/users	Список учетных записей (администратор)
POST	/users	Создать студента или преподавателя
PATCH	/users/{user_id}	Редактировать учетную запись
GET	/olympiads	Все олимпиады, фильтр по году
POST	/olympiads	Создать олимпиаду
GET	/olympiads/{olympiad_id}	Карточка олимпиады
PATCH	/olympiads/{olympiad_id}	Изменить поля карточки
PUT	/olympiads/{olympiad_id}	Заменить данные карточки
DELETE	/olympiads/{olympiad_id}	Удалить пустую олимпиаду
POST	/olympiads/{olympiad_id}/participations	Записаться на олимпиаду

Метод	Путь	Назначение
GET	/olympiads/{olympiad_id}/participations	Заявки для ответственного преподавателя
GET	/users/me/participations	Свои заявки и опубликованные результаты
PUT	/participations/{participation_id}/result	Записать баллы и результат
DELETE	/participations/{participation_id}	Удалить заявку до подготовки протокола
GET	/protocols	Состояния протоколов (служебный список)
GET	/olympiads/{olympiad_id}/protocol	Опубликованный протокол (общедоступный)
PATCH	/olympiads/{olympiad_id}/protocol	Подготовить, отозвать или опубликовать протокол
GET	/olympiads/{olympiad_id}/protocol-draft	Просмотреть служебный протокол
GET	/olympiads/{olympiad_id}/reports/protocol	Скачать протокол Word или Excel
GET	/reports/annual	Годовой отчёт по опубликованным результатам
GET	/olympiads/{olympiad_id}/attachments	Положение и бланки; задания прошлых лет
POST	/olympiads/{olympiad_id}/attachments	Прикрепить файл
GET	/attachments/{attachment_id}	Скачать общедоступный документ
PUT	/attachments/{attachment_id}	Заменить файл или название
DELETE	/attachments/{attachment_id}	Удалить документ
GET	/olympiads/{olympiad_id}/attachments/manage	Все файлы ответственного преподавателя
POST	/olympiads/{olympiad_id}/templates	Создать положение или бланк Word из текста

3.5.1 Примеры запросов и обработка ответа

Пример клиентского обмена приведён в приложении 5. В отличие от отдельных слайдов, response.json вызывается с круглыми скобками. До разбора ответа проверяется код или вызывается raise_for_status(). Каждый сетевой запрос ограничен таймаутом.

Для создания карточки администратор передаёт название, описание, teacherId и даты. teacherId должен указывать на активного преподавателя. Для создания студента передаются все поля из ТЗ. Профиль и административное редактирование имеют разные схемы, поэтому участник не может присвоить себе роль администратора.

Файлы передаются multipart/form-data, обычные данные — JSON. Экспорт возвращает двоичный ответ, который сохраняется через response.content. DELETE при успехе возвращает 204 с пустым телом. Попытка вызвать json() для такого ответа является ошибкой клиента, а не требованием сервиса.

Бизнес-ошибки возвращаются в detail. Конфликт уникальности и недопустимый переход состояния имеют код 409. Неверный формат данных — 422, превышение лимита файла — 413. Полный справочник параметров и обязательных полей содержится в OpenAPI и API.md.

3.5.2 Клиент первого этапа и граница с Django

В первом этапе подготовлен консольный requests-клиент. Он умеет показывать олимпиады, карточку, публичный протокол и документы, выполнять вход, читать профиль и свои результаты, подавать заявку. Сложные служебные действия доступны через Swagger и описаны в руководстве пользователя.

Клиент разделён на config.py, api/accounts.py, api/olympiads.py и main.py. Пользовательский ввод находится в main, а функции API получают параметры и возвращают Response. Сетевые ошибки перехватываются через requests.RequestException, пароль вводится без отображения через getpass.

Django-приложение с шаблонами, формами и панелью администратора на первом этапе не реализуется. При его создании данные должны запрашиваться серверным Python-кодом через этот API, как требует методичка. Не следует дублировать запись результатов прямыми запросами к таблицам, обходя проверку протокола.

Отсутствующие на этом этапе Django-страницы не включены в отчёт как проверенные. Браузерные снимки относятся к работающим Swagger и ReDoc. Сценарий демонстрации, порядок смены ролей и пояснения к устройству кода находятся в DEFENSE.md.

3.5.3 Руководство по развёртыванию

Для запуска нужны Python 3.12, Docker с Compose v2 и свободные порты 5432 и 8000. Python работает в виртуальном окружении, PostgreSQL — отдельным контейнером. Установка выполняется из корня проекта по README. Сначала устанавливаются зафиксированные зависимости, затем создаётся .env со случайным JWT-ключом.

Команда docker compose up -d --wait ждёт готовности БД. create_test_db.py создаёт отдельную базу для тестов. app.init_db --demo создаёт таблицы и вымышленные данные только в пустой базе. Повторная команда не очищает записи. Для новой базы без примеров используется init_db без --demo и ADMIN_PASSWORD из настроек.

После запуска Uvicorn выполняется GET /health и представительный запрос списка. Проверка существования процесса или открытого порта недостаточна. Затем следует вход и запрос профиля, а для проверки всего пути — scripts.smoke. Последний создаёт отдельную демонстрационную олимпиаду и сохраняет пример результата.

Подробные команды Windows/Linux, значения переменных, остановка и диагностика находятся в DEPLOYMENT.md. Файлы .env и .venv не передаются в архиве. Приложение не требует внешних API или платных ключей. Секрет подписи генерируется локально и не хранится в Git.

3.5.4 Хранение данных и резервное копирование

Информация находится в двух местах: строки — в PostgreSQL volume, документы — в storage. Для восстановления нужны обе части одного состояния. Копирование только исходников не сохраняет пользовательские загрузки, а копирование только storage не восстанавливает заявки и права.

Перед согласованным резервным копированием Uvicorn останавливается. База выгружается pg_dump в custom-формате, storage архивируется отдельно. Восстановление выполняется в новую пустую базу, чтобы не затронуть рабочую. После него проверяются вход, список событий и скачивание нескольких файлов.

В учебном Compose база доступна только через loopback. Для эксплуатации с реальными персональными данными потребуются отдельные настройки TLS, доступа, резервного хранения и ограничения частоты входа. Нагрузочная пригодность для большого числа пользователей данным набором проверок не подтверждена.

Скрипт create_all создаёт отсутствующие таблицы, но не изменяет уже существующие столбцы. При дальнейшем изменении модели нужен отдельный процесс миграции. Автоматическое удаление базы и перезаполнение не применяется как способ обновления проекта.

3.6 Результаты тестирования

Фактически выполнено 75 автоматических тестов: успешно 75, падений 0, пропусков 0. Покрытие строк app — 89.31%, модулей API — 99.27% (406 из 409). Прогон выполнен 19 октября 2026 года; точное время и длительность приведены в junit.xml и pytest.txt.

Тесты проверили операции учётных записей, карточки, отбор и версии списка, роли, принадлежность преподавателя, заявки, результаты, переходы статусов, документы и экспорт. В бинарных отчётах проверено фактическое содержимое таблиц, а не только HTTP-код. В Excel отдельно проверено сохранение строки, начинающейся со знака формулы, в безопасном текстовом виде.

Через работающий HTTP-сервер выполнен сценарий от создания студента до публикации и скачивания четырёх отчётов. До публикации запрос гостя возвращал 404, после — результат winner и 95.5 балла. Открыты Swagger v1/v2 и ReDoc, выполнен запрос из Swagger. Снимки относятся к реальному запуску.

Общая доля проверенного кода ниже доли маршрутов, поскольку CLI первоначального наполнения проверялся отдельно и не исполнялся pytest. Проценты не означают отсутствия всех ошибок. Не проводились нагрузочные испытания, проверки многодневной устойчивости и тестирование Django-приложения.

Первоначальные ошибки подключения тестовой БД у подприложений и маршрутов v2 устранены. Также закреплена совместимая версия Pydantic. Реестр содержит только наблюдавшиеся проблемы, их воспроизведение и исправления. Полный отчёт, фикстуры, сценарии и матрица — в TESTING.md и приложениях.

Фактический результат pytest · 19.10.2026

75 / 75 успешно · ошибок 0 · пропусков 0
Покрытие app: 89.31% · API: 99.27%

```
.................................................................... [ 96%]
...                                                                  [100%]
----- generated xml file: /workspace/kursach_penis/docs/evidence/junit.xml -----
============================== tests coverage ==============================
_________________ coverage: platform linux, python 3.12.14-final-0 _________________

Name                              Stmts   Miss  Cover   Missing
---------------------------------------------------------------
app/__init__.py                       0      0   100%
app/api/__init__.py                   0      0   100%
app/api/attachments.py              128      3    98%   71-73
app/api/auth.py                      39      0   100%
app/api/olympiads.py                 68      0   100%
app/api/participations.py            66      0   100%
app/api/protocols.py                 68      0   100%
app/api/users.py                     40      0   100%
app/config.py                        10      0   100%
app/database.py                      12      3    75%   11, 18-19
app/init_db.py                       85     85     0%   2-110
app/main.py                          47      1    98%   25
app/models/__init__.py                0      0   100%
app/models/attachment.py             10      0   100%
app/models/olympiad.py               12      0   100%
app/models/participation.py          10      0   100%
app/models/user.py                   18      0   100%
app/reports.py                       42      0   100%
app/schemas/__init__.py               0      0   100%
app/schemas/account_update.py         5      0   100%
app/schemas/api_model.py              4      0   100%
app/schemas/attachment_read.py        7      0   100%
app/schemas/olympiad_create.py       14      0   100%
app/schemas/olympiad_page.py          7      0   100%
app/schemas/olympiad_read.py         10      0   100%
app/schemas/olympiad_update.py       14      0   100%
app/schemas/password_change.py        5      0   100%
app/schemas/profile_update.py        19      0   100%
app/schemas/protocol_update.py        4      0   100%
app/schemas/result_update.py          7      0   100%
app/schemas/user_create.py           22      0   100%
app/schemas/user_read.py             14      0   100%
app/security.py                      46      0   100%
app/services.py                      28      0   100%
---------------------------------------------------------------
TOTAL                               861     92    89%
Coverage HTML written to dir docs/evidence/coverage
Coverage JSON written to file docs/evidence/coverage.json
75 passed in 3.01s
```

Рисунок 7 — Отчёт фактического автоматического прогона

4 ГЛОССАРИЙ

В таблице 7 приведены термины и сокращения, используемые в работе.

Таблица 7 — Термины и сокращения

Термин	**Определение**
APM	Автоматизированное рабочее место.
API	Программный интерфейс для взаимодействия приложений.
REST	Подход к организации HTTP-взаимодействия с ресурсами.
HTTP	Протокол запросов и ответов клиента и сервера.
JSON	Текстовый формат обмена структурированными данными.
JWT	Подписанный токен с набором утверждений и сроком действия.
ORM	Отображение объектов программы на таблицы базы данных.
БД / СУБД	База данных / система управления базами данных.
PK / FK	Первичный ключ / внешний ключ.
CRUD	Создание, чтение, обновление и удаление записей.
OpenAPI	Машиночитаемое описание операций HTTP API.
Swagger / ReDoc	Интерфейсы чтения и проверки OpenAPI.
PBKDF2	Функция получения хеша пароля с солью и повторными вычислениями.
Транзакция	Группа операций с подтверждением или откатом.
Пагинация	Получение списка частями заданного размера.
Фикстура	Подготовка и освобождение состояния теста.
Покрытие	Доля строк или ветвей кода, выполненных при проверках.
TDD	Разработка через цикл тест — реализация — рефакторинг.

5 СПИСОК ИСТОЧНИКОВ

1. Техническое задание «Пеннер (1)». Тема: создание и тестирование АРМ учёта участия студентов в олимпиадном движении. Предоставленный материал, 2026.

2. Методические рекомендации по выполнению курсового проекта ПМ.05 / сост. А. В. Грибова. Екатеринбург: ГАПОУ СО «ЕМК», 2026. Предоставленный PDF.

3. Общие правила оформления курсовой работы. Раздел 3. Предоставленный PDF, без выходных данных.

4. Рамки (1). Шаблон оформления курсового проекта. Предоставленный DOCX, без выходных данных.

5. Библиотека requests. Презентация о HTTP, REST и запросах Python. Предоставленный PDF, без выходных данных.

6. FastAPI. Документация разработчика: маршруты, зависимости, безопасность, подприложения. Электронный ресурс: https://fastapi.tiangolo.com/.

7. Pydantic. Документация версии 2: модели, поля, валидация. Электронный ресурс: https://docs.pydantic.dev/latest/.

8. SQLAlchemy. Документация версии 2: ORM, сессии и транзакции. Электронный ресурс: https://docs.sqlalchemy.org/en/20/.

9. PostgreSQL Global Development Group. PostgreSQL 17 Documentation. Электронный ресурс: https://www.postgresql.org/docs/17/.

10. Requests. User Guide. Электронный ресурс: https://requests.readthedocs.io/en/latest/.

11. PyJWT. Documentation: encoding and decoding tokens. Электронный ресурс: https://pyjwt.readthedocs.io/en/stable/.

12. python-docx. Documentation: documents and tables. Электронный ресурс: https://python-docx.readthedocs.io/en/latest/.

13. openpyxl. Documentation: workbook and cell values. Электронный ресурс: https://openpyxl.readthedocs.io/en/stable/.

14. pytest. Documentation: fixtures and parametrization. Электронный ресурс: https://docs.pytest.org/en/stable/.

15. Fielding R., Nottingham M., Reschke J. HTTP Semantics. RFC 9110. IETF, 2022. Электронный ресурс: https://www.rfc-editor.org/rfc/rfc9110.

ЗАКЛЮЧЕНИЕ

В рамках первого этапа разработан REST-сервис для обработки сведений об участии студентов в олимпиадах и конкурсах профессионального мастерства. Определены четыре роли и модель PostgreSQL из четырёх связанных таблиц. Реализованы создание учётных записей, карточки мероприятий, заявки, результаты, документы и формирование Word/Excel.

Публикация отделена от подготовки протокола. До неё гость и студент не получают незавершённые результаты. Преподаватель ограничен своими мероприятиями, администратор управляет учётными записями и публикует подготовленные итоги. Валидация и ограничения БД препятствуют некорректным датам, повторным заявкам и ряду ошибочных переходов.

Созданы версии v1 и v2, отдельные Swagger/ReDoc, снимки OpenAPI и учебный requests-клиент. Автоматический набор и HTTP-сценарий подтвердили выбранные функции первого этапа. Предоставлены воспроизводимое наполнение, примеры файлов, инструкция установки, тестовая документация и материалы для демонстрации.

Цель разработки REST-сервиса достигнута в проверенном учебном окружении. Следующим этапом является самостоятельное Django-приложение с серверными шаблонами и формами, которое использует разработанный контракт. Перед формальной сдачей необходимо заполнить неизвестные реквизиты, согласовать расхождения оформления и состав библиографии. Эти сведения не подменены вымышленными значениями.

ПРИЛОЖЕНИЕ 1 — РЕЕСТР ДЕФЕКТОВ И ОГРАНИЧЕНИЙ

Таблица 8 — Выявленные проблемы

ID	Проблема	Важность	Шаги и факт	Исправление / статус
B01	При прямом добавлении маршрутов v2 не применялась подмена get_db	Средняя, тестируемость	Запустить test_v2_security_and_write: ожидалось 201, получено 401	Общий APIRouter включается через include_router; тест проходит; исправлено
B02	Неограниченная транзитивная Pydantic 2.14.1 вызывала предупреждения совместимости	Низкая	Первый прогон выдавал UnsupportedFieldAttributeWarning при FastAPI 0.115.12	Зафиксированы Pydantic 2.11.7 и совместимый AnyIO, повтор без предупреждений; исправлено
E01	Фикстура первоначально подменяла БД только у корневого приложения	Средняя, тестовая инфраструктура	Защищённые запросы тестов возвращали 401, публичный список пуст	Подмена зависимости настроена у обоих подприложений; исправлено

Не создавались вымышленные дефекты предметной логики, результаты нагрузочных испытаний или статистика многодневной стабильности. Первые проверки не проходили, финальный отчёт относится к исправленному состоянию. v2 добавляет пагинацию и поиск; ошибки B01/B02 исправлены в общей реализации, v1 намеренно не оставлена с ошибками.

Приложение фиксирует исправленные проблемы разработки и тестовой инфраструктуры. Оно не заменяет окончательную приёмку и не содержит вымышленных дефектов. Настройки защищённого производственного размещения и второй этап Django в проверенный объём не входят.

ПРИЛОЖЕНИЕ 2 — ТЕСТОВЫЕ СЦЕНАРИИ

TC01 — R04

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Верный логин, просмотр профиля, выход. Ожидается: 200; профиль без хеша; прежний токен после выхода — 401.

Реализация: test_login_profile_logout. Фактический итог финального прогона: успешно.

TC02 — R04

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Неверный логин/пароль, заблокированный аккаунт. Ожидается: 401 без выдачи токена.

Реализация: test_bad_login; test_inactive_login. Фактический итог финального прогона: успешно.

TC03 — R04

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Повреждённый, пустой, просроченный JWT, несуществующий sub. Ожидается: 401.

Реализация: test_invalid_token; test_missing_user_and_expired_token. Фактический итог финального прогона: успешно.

TC04 — R04

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Изменить email и ФИО, попытаться изменить role; сменить пароль. Ожидается: 200; роль — 422; старый токен — 401.

Реализация: test_profile_and_password. Фактический итог финального прогона: успешно.

TC05 — R07 R13

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Создать студента администратором, преподавателем, студентом. Ожидается: 201 / 201 / 403.

Реализация: test_create_student. Фактический итог финального прогона: успешно.

TC06 — R07

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Создать преподавателя разными ролями; повторить запись. Ожидается: 403 / 201; повтор — 409.

Реализация: test_create_teacher_and_duplicate. Фактический итог финального прогона: успешно.

TC07 — R13

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Неверные email, курс, пароль, логин, дата, отсутствующие данные. Ожидается: 422 для каждой из девяти комбинаций.

Реализация: test_user_validation. Фактический итог финального прогона: успешно.

TC08 — R07

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Список аккаунтов, блокировка, смена пароля, дубликат email. Ожидается: 200; отзыв токенов; самоблокировка — 409; конфликт email — 409.

Реализация: test_admin_accounts; test_duplicate_profile_email. Фактический итог финального прогона: успешно.

TC09 — R08

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Создать, получить, PATCH, PUT и удалить пустую олимпиаду. Ожидается: 201 / 200 / 200 / 200 / 204; затем 404.

Реализация: test_crud. Фактический итог финального прогона: успешно.

TC10 — R08

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Изменять карточку студентом/преподавателем. Ожидается: 403.

Реализация: test_crud_forbidden. Фактический итог финального прогона: успешно.

TC11 — R01 R16

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Год, поиск, страницы, проценты в строке поиска, неверные лимиты. Ожидается: Отбор точен; разные контракты; некорректные пределы — 422.

Реализация: test_filter_and_versions; test_pagination_validation. Фактический итог финального прогона: успешно.

TC12 — R08

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Неверные даты, NULL, студент/неактивный ответственный. Ожидается: 422.

Реализация: test_invalid_olympiad; test_inactive_teacher. Фактический итог финального прогона: успешно.

TC13 — R05 R06 R09 R14

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Запись → результат → подготовка → отзыв → подготовка → публикация. Ожидается: До публикации 404/null; после — баллы и winner; правки заблокированы.

Реализация: test_complete_workflow. Фактический итог финального прогона: успешно.

TC14 — R09 R14

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Чужой преподаватель читает/меняет протокол, баллы, заявки. Ожидается: 403; в списке /protocols только свои олимпиады.

Реализация: test_ownership. Фактический итог финального прогона: успешно.

TC15 — R05

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Повторная заявка, отмена, чужая заявка, прошедшая дата. Ожидается: 409; своя отмена 204; чужая 403; прошедшая олимпиада 409.

Реализация: test_enrollment_and_deletion. Фактический итог финального прогона: успешно.

TC16 — R14

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Пустой/неполный протокол, будущая дата, отрицательные и дробные баллы. Ожидается: 409 для состояния; 422 для неверных данных.

Реализация: test_protocol_incomplete_future_and_validation. Фактический итог финального прогона: успешно.

TC17 — R10 R11

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Открыть DOCX/XLSX; пустой год, черновой протокол, неизвестный формат. Ожидается: Настоящие таблицы; score=95.5; пустой отчёт допустим; формат pdf — 422.

Реализация: test_complete_workflow; test_draft_export. Фактический итог финального прогона: успешно.

TC18 — R10

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: ФИО начинается со знака формулы Excel. Ожидается: Ячейка строковая, формула не исполняется.

Реализация: test_excel_formula_safety. Фактический итог финального прогона: успешно.

TC19 — R03 R15

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Загрузить файл, скачать, заменить, удалить. Ожидается: 201 / 200 / 200 / 204; затем 404.

Реализация: test_upload_download_replace_delete. Фактический итог финального прогона: успешно.

TC20 — R03

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Текущее задание, затем изменить год события на прошлый. Ожидается: Текущее скрыто; прошлогоднее доступно.

Реализация: test_past_tasks. Фактический итог финального прогона: успешно.

TC21 — R12 R15

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Создать четыре вида Word; две грамоты одной олимпиады; чужой преподаватель. Ожидается: Каждый шаблон 201; две записи сохранены; чужой доступ 403.

Реализация: test_templates; test_multiple_awards_and_permissions. Фактический итог финального прогона: успешно.

TC22 — R03

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Файл 5 МБ+1, неверные сигнатуры/расширения/кодировка, корректные DOCX/XLSX. Ожидается: 413 / 422; корректные документы — 201.

Реализация: test_file_validation; test_office_uploads. Фактический итог финального прогона: успешно.

TC23 — R03

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Удалить физический файл; передать ../../ в имени. Ожидается: 404; путь хранения остаётся в каталоге и имеет UUID.

Реализация: test_missing_file; test_no_path_traversal. Фактический итог финального прогона: успешно.

TC24 — R16

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: CRUD через v2, чужая роль, Swagger и ReDoc обеих версий. Ожидается: Права совпадают с v1; схемы и локальные страницы 200.

Реализация: test_v2_security_and_write; test_documentation_and_security_schema. Фактический итог финального прогона: успешно.

TC25 — R17 R18

Предусловия: тестовая транзакция PostgreSQL; аккаунты и олимпиада из фикстур.

Действия и данные: Проверить SELECT 1 и локальные CSS. Ожидается: 200; database=postgresql.

Реализация: test_health_and_static. Фактический итог финального прогона: успешно.

ПРИЛОЖЕНИЕ 3 — МАТРИЦА ТРАССИРОВКИ

Таблица 9 сопоставляет требования R01–R19 из REQUIREMENTS.md и семейства тестов приложения 2. Порог заполнения данных проверен отдельно, а не выдан за автотест функциональности.

Таблица 9 — Требования и проверки

Требование	**Семейства тестов**
R01	TC11
R02	
R03	TC19, TC20, TC22, TC23
R04	TC01, TC02, TC03, TC04
R05	TC13, TC15
R06	TC13
R07	TC05, TC06, TC08
R08	TC09, TC10, TC12
R09	TC13, TC14
R10	TC17, TC18
R11	TC17
R12	TC21
R13	TC05, TC07
R14	TC13, TC14, TC16
R15	TC19, TC21
R16	TC11, TC24
R17	TC25
R18	TC25
R19	Проверка seed_counts.json; повторный запуск --demo

ПРИЛОЖЕНИЕ 4 — ОСНОВНЫЕ ЛИСТИНГИ

Показаны код безопасности, управление состоянием и небольшой requests-клиент. Полные исходники находятся в app/ и client/. Это рабочие файлы комплекта; номера строк добавлены только для чтения листинга.

app/security.py (строки 1–50)

```
  1 """Пароли хранятся в виде PBKDF2-хеша с отдельной солью."""
  2 from datetime import datetime, timedelta, timezone
  3 import hashlib
  4 import hmac
  5 import secrets
  6 import jwt
  7 from fastapi import Depends, HTTPException
  8 from fastapi.security import OAuth2PasswordBearer
  9 from sqlalchemy.orm import Session
 10 from app.config import JWT_SECRET, TOKEN_MINUTES
 11 from app.database import get_db
 12 from app.models.user import User
 13
 14 oauth2 = OAuth2PasswordBearer(tokenUrl="auth/token")
 15
 16
 17 def hash_password(password):
 18     salt = secrets.token_hex(16)
 19     value = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(),
600000)
 20     return f"pbkdf2_sha256${salt}${value.hex()}"
 21
 22
 23 def verify_password(password, stored):
 24     try:
 25         method, salt, expected = stored.split("$")
 26         if method != "pbkdf2_sha256":
 27             return False
 28         value = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(),
600000)
 29         return hmac.compare_digest(value.hex(), expected)
 30     except (ValueError, TypeError):
 31         return False
 32
 33
 34 def create_token(user):
 35     now = datetime.now(timezone.utc)
 36     return jwt.encode({"sub": str(user.id), "ver": user.token_version,
 37                        "iat": now, "exp": now + timedelta(minutes=TOKEN_MINUTES)},
 38                       JWT_SECRET, algorithm="HS256")
 39
 40
 41 def get_current_user(token: str = Depends(oauth2), db: Session = Depends(get_db)):
 42     error = HTTPException(401, "Войдите в систему повторно",
 43                           headers={"WWW-Authenticate": "Bearer"})
 44     try:
 45         payload = jwt.decode(token, JWT_SECRET, algorithms=["HS256"],
 46                              options={"require": ["sub", "exp", "iat", "ver"]})
 47         user = db.get(User, int(payload["sub"]))
 48     except (jwt.InvalidTokenError, ValueError, TypeError, KeyError):
 49         raise error
 50     if not user or not user.is_active or payload["ver"] != user.token_version:
```

app/security.py (строки 51–65)

```
51         raise error
52     return user
53
54
55 def require_staff(user: User = Depends(get_current_user)):
56     if user.role not in ("admin", "teacher"):
57         raise HTTPException(403, "Нужны права преподавателя или администратора")
58     return user
59
60
61 def require_admin(user: User = Depends(get_current_user)):
62     if user.role != "admin":
63         raise HTTPException(403, "Нужны права администратора")
64     return user
65
```

app/api/protocols.py (строки 1–50)

```
  1 from datetime import date
  2 from fastapi import APIRouter, Depends, HTTPException, Query
  3 from sqlalchemy import extract, or_
  4 from sqlalchemy.orm import Session
  5 from app.database import get_db
  6 from app.models.user import User
  7 from app.models.olympiad import Olympiad
  8 from app.models.participation import Participation
  9 from app.schemas.protocol_update import ProtocolUpdate
 10 from app.schemas.olympiad_read import OlympiadRead
 11 from app.security import require_admin, require_staff
 12 from app.services import check_owner, get_olympiad, save
 13 from app.api.participations import participation_data
 14 from app.reports import RESULT_NAMES, make_report
 15
 16 router = APIRouter(tags=["Протоколы и отчёты"])
 17
 18
 19 def protocol_rows(db, olympiad_id):
 20     return db.query(Participation, User).join(User, User.id ==
Participation.student_id).filter(Participation.olympiad_id ==
olympiad_id).order_by(Participation.score.desc().nullslast(), Participation.id).all()
 21
 22
 23 def protocol_data(db, item):
 24     return {"olympiadId": item.id, "title": item.title, "status":
item.protocol_status,
 25             "participants": [participation_data(p, s) for p, s in
protocol_rows(db, item.id)]}
 26
 27
 28 @router.get("/protocols", response_model=list[OlympiadRead], summary="Состояния
протоколов (служебный список)")
 29 def list_protocols(user: User = Depends(require_staff), db: Session =
Depends(get_db)):
 30     query = db.query(Olympiad)
 31     if user.role == "teacher":
 32         query = query.filter(Olympiad.teacher_id == user.id)
 33     return query.order_by(Olympiad.id).all()
 34
 35
 36 @router.get("/olympiads/{olympiad_id}/protocol", summary="Опубликованный протокол
(общедоступный)")
 37 def public_protocol(olympiad_id: int, db: Session = Depends(get_db)):
 38     item = get_olympiad(db, olympiad_id)
 39     if item.protocol_status != "published":
 40         raise HTTPException(404, "Протокол ещё не опубликован")
 41     return protocol_data(db, item)
 42
 43
 44 @router.get("/olympiads/{olympiad_id}/protocol-draft", summary="Просмотреть
служебный протокол")
 45 def private_protocol(olympiad_id: int, user: User = Depends(require_staff),
 46                      db: Session = Depends(get_db)):
 47     item = get_olympiad(db, olympiad_id)
 48     check_owner(item, user)
 49     return protocol_data(db, item)
 50
```

app/api/protocols.py (строки 51–95)

```
 51
 52 @router.patch("/olympiads/{olympiad_id}/protocol", summary="Подготовить, отозвать
или опубликовать протокол")
 53 def set_protocol_status(olympiad_id: int, data: ProtocolUpdate,
 54                           user: User = Depends(require_staff), db: Session =
Depends(get_db)):
 55     item = get_olympiad(db, olympiad_id, lock=True)
 56     check_owner(item, user)
 57     current = item.protocol_status
 58     if data.status == "published" and user.role != "admin":
 59         raise HTTPException(403, "Публикует только администратор")
 60     if (current, data.status) not in [("draft", "prepared"), ("prepared",
"draft"), ("prepared", "published")]:
 61         raise HTTPException(409, "Недопустимый переход статуса протокола")
 62     if data.status == "prepared":
 63         if item.end_date > date.today():
 64             raise HTTPException(409, "Олимпиада ещё не завершена")
 65         query = db.query(Participation).filter_by(olympiad_id=item.id)
 66         if not query.count() or query.filter(or_(Participation.score.is_(None),
Participation.result.is_(None))).count():
 67             raise HTTPException(409, "Укажите баллы и результат каждого
участника")
 68     item.protocol_status = data.status
 69     save(db)
 70     return {"status": item.protocol_status}
 71
 72
 73 @router.get("/olympiads/{olympiad_id}/reports/protocol", summary="Скачать протокол
Word или Excel")
 74 def export_protocol(olympiad_id: int, file_format: str = Query(default="docx",
alias="format"),
 75                       user: User = Depends(require_staff), db: Session =
Depends(get_db)):
 76     item = get_olympiad(db, olympiad_id)
 77     check_owner(item, user)
 78     rows = []
 79     for number, (part, student) in enumerate(protocol_rows(db, item.id), 1):
 80         rows.append([number, student.full_name, float(part.score) if part.score is
not None else "—",
 81                      RESULT_NAMES.get(part.result, "Не определён")])
 82     title = f"Протокол: {item.title}. Статус: {item.protocol_status}"
 83     return make_report(title, ["№", "ФИО", "Баллы", "Результат"], rows,
file_format, f"protocol_{item.id}")
 84
 85
 86 @router.get("/reports/annual", summary="Годовой отчёт по опубликованным
результатам")
 87 def annual_report(year: int = Query(ge=2000, le=2100),
 88                    file_format: str = Query(default="xlsx", alias="format"),
 89                    user: User = Depends(require_admin), db: Session =
Depends(get_db)):
 90     rows = db.query(Olympiad, Participation, User).join(Participation,
Participation.olympiad_id == Olympiad.id).join(User, User.id ==
Participation.student_id).filter(extract("year", Olympiad.start_date) == year,
Olympiad.protocol_status == "published").order_by(Olympiad.start_date, Olympiad.id,
Participation.id).all()
 91     data = [[o.title, o.start_date.isoformat(), s.full_name, float(p.score),
RESULT_NAMES[p.result]] for o, p, s in rows]
 92     return make_report(f"Участие студентов в олимпиадах за {year} год. Записей
участия: {len(data)}",
 93                        ["Олимпиада", "Дата", "ФИО", "Баллы", "Результат"], data,
 94                        file_format, f"annual_{year}")
 95
```

client/api/accounts.py (строки 1–19)

```
  1 import requests
  2 from client.config import BASE_URL, TIMEOUT
  3
  4
  5 def login(username, password):
  6     return requests.post(f"{BASE_URL}/auth/token", data={"username": username,
"password": password}, timeout=TIMEOUT)
  7
  8
  9 def get_profile(token):
 10     return requests.get(f"{BASE_URL}/users/me", headers={"Authorization": f"Bearer
{token}"}, timeout=TIMEOUT)
 11
 12
 13 def get_results(token):
 14     return requests.get(f"{BASE_URL}/users/me/participations",
headers={"Authorization": f"Bearer {token}"}, timeout=TIMEOUT)
 15
 16
 17 def enroll(olympiad_id, token):
 18     return requests.post(f"{BASE_URL}/olympiads/{olympiad_id}/participations",
headers={"Authorization": f"Bearer {token}"}, timeout=TIMEOUT)
 19
```

ПРИЛОЖЕНИЕ 5 — ЗАПРОСЫ И ПРИМЕРЫ РЕЗУЛЬТАТОВ

Ниже показаны реальные страницы документации и примеры файлов из HTTP-сценария. Токены и значения JWT_SECRET на снимках не отображаются. Файлы protocol.docx/xlsx и annual.docx/xlsx находятся в docs/examples/.

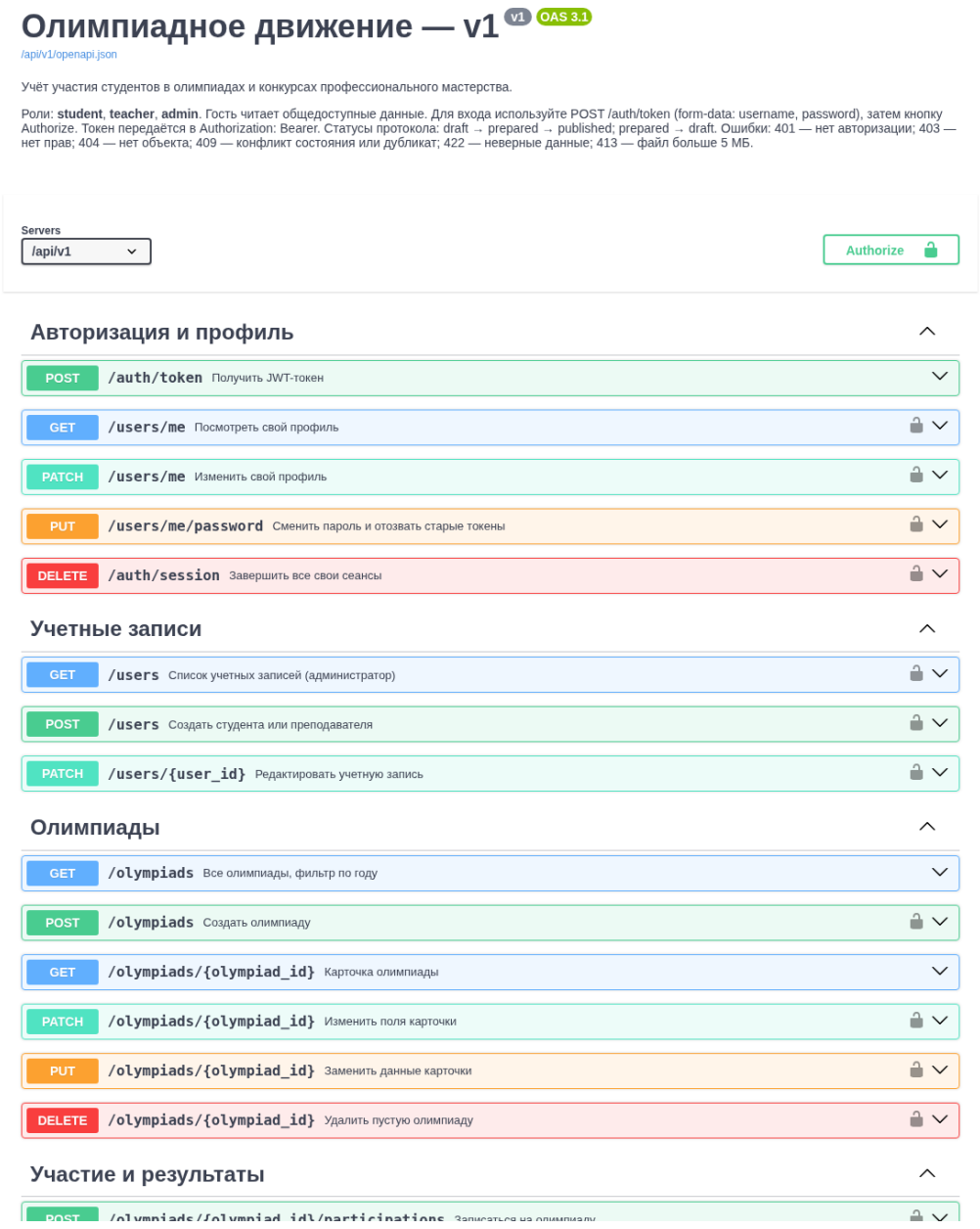


Рисунок 8 — Swagger первой версии работающего сервиса

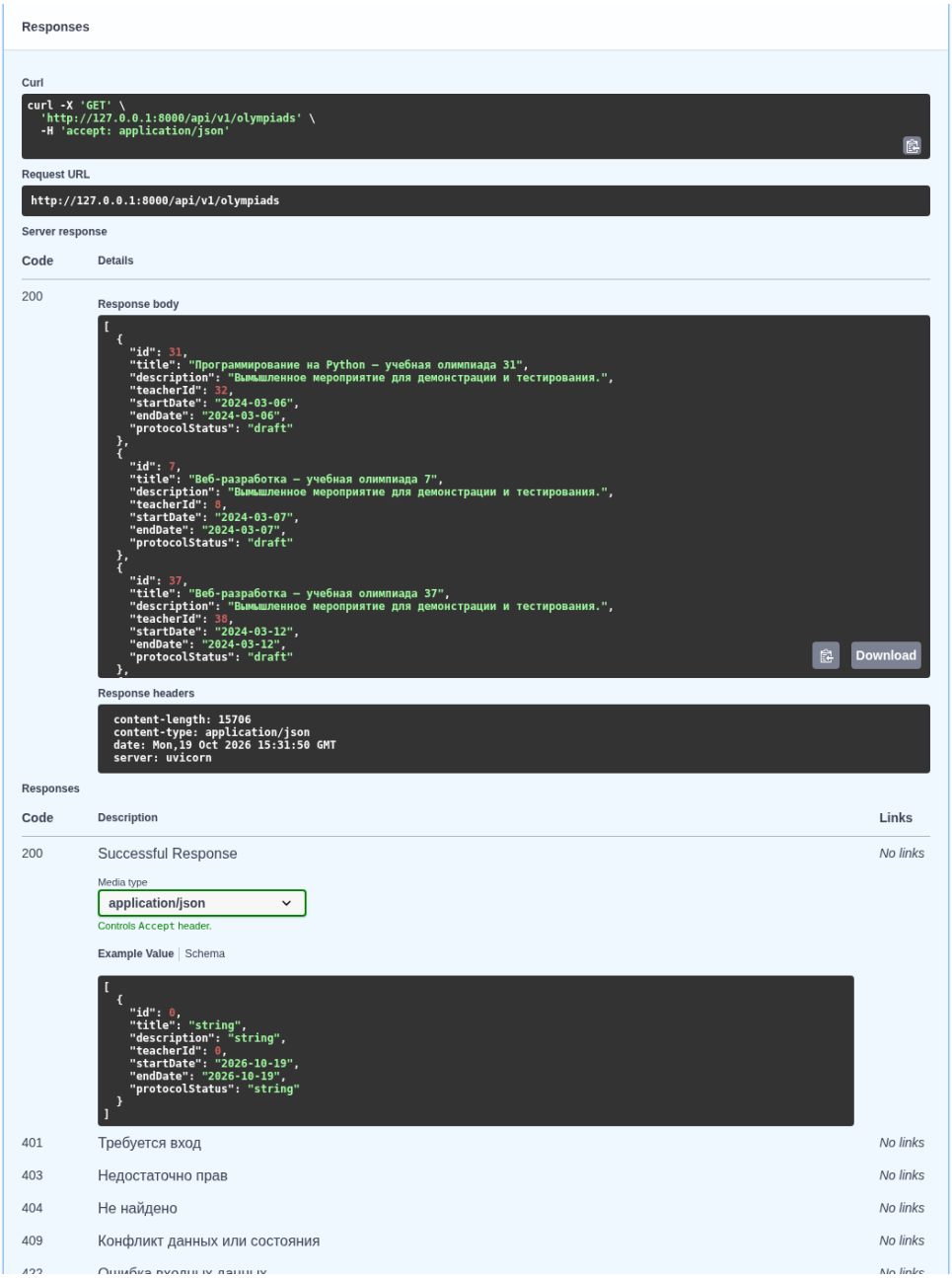


Рисунок 9 — Выполнение GET-запроса из Swagger

Публичные запросы выполняются без авторизации. Для изменяющих операций необходимо войти через Authorize. Код ответа, заголовки и тело позволяют проверить фактический результат, как в презентации requests.

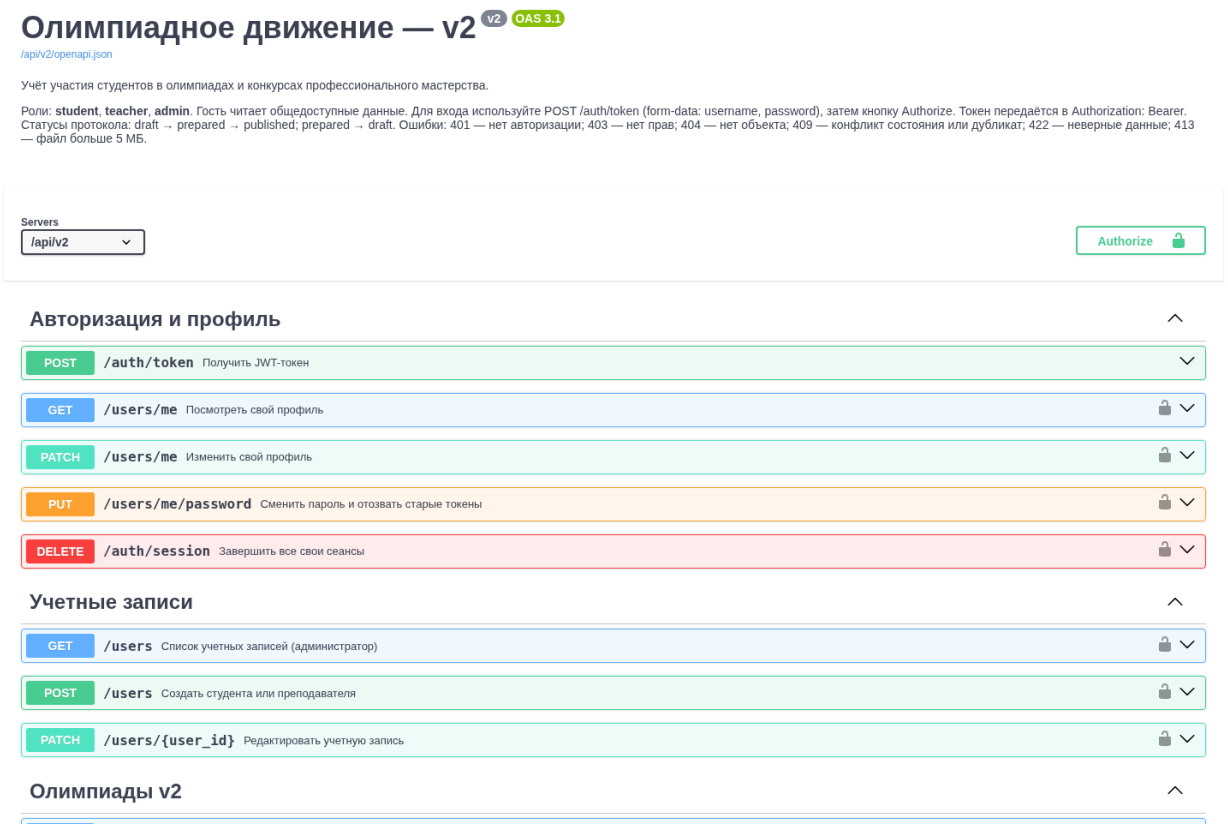


Рисунок 10 — Документация второй версии

Вторая версия использует тот же набор предметных операций. Отличие списка: объект с items и параметрами страницы. Полный JSON-контракт и примеры запросов приведены в API.md.

Фактический HTTP-сценарий: 2026-10-19T15:22:36.980377+00:00. Результат: passed. Проверены вход, создание студента и олимпиады, заявка, баллы, подготовка, публикация и четыре бинарные выгрузки.

Таблица 10 — Содержимое примера опубликованного протокола

№	ФИО	Баллы	Результат
1	Учебный Студент Для Проверки	95.5	Победитель

```
import requests
response = requests.get(
    "http://127.0.0.1:8000/api/v1/olympiads",
    params={"year": 2026}, timeout=10
)
response.raise_for_status()
print(response.status_code)
print(response.json())
```

В примере ответ Response не подменяется словарём: сначала проверяется успешность, затем вызывается json(). Для сохранения отчёта используется content. Больше примеров и полный сценарий доступны в scripts/smoke.py и client/.

ПРИЛОЖЕНИЕ 6 — РЕКОМЕНДУЕМАЯ ЛИТЕРАТУРА

Перечень книг для дополнительного самостоятельного изучения. Он отделён от технических источников основной части: прочтение всех книг при подготовке комплекта не заявляется. Книги не заменяют требуемые методичкой материалы периодической печати.

16. Beaulieu A. Learning SQL. 3rd ed. O’Reilly Media, 2020.

17. Fowler M. Refactoring: Improving the Design of Existing Code. 2nd ed. Addison-Wesley, 2018.

18. Grinberg M. Flask Web Development. 2nd ed. O’Reilly Media, 2018.

19. Kleppmann M. Designing Data-Intensive Applications. O’Reilly Media, 2017.

20. Lubanovic B. Introducing Python. 2nd ed. O’Reilly Media, 2019.

21. Martin R. C. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Prentice Hall, 2017.

22. Matthes E. Python Crash Course. 3rd ed. No Starch Press, 2023.

23. Percival H. J. W. Test-Driven Development with Python. 2nd ed. O’Reilly Media, 2017.

24. Ramalho L. Fluent Python. 2nd ed. O’Reilly Media, 2022.

25. Richards M., Ford N. Fundamentals of Software Architecture. O’Reilly Media, 2020.